

W Y K Ł A D 1

Świat Pythona: ekosystem i filozofia

Wprowadzenie do programowania II

Informatyka Stosowana | semestr letni 2025/2026

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- 1 Historia i popularność Pythona
- 2 Interpreter, CPython, Zen of Python
- 3 Instalacja, pip, wirtualne środowiska
- 4 Narzędzia: VS Code, Jupyter, Colab, AI
- 5 PEP 8 i konwencje kodowania
- 6 Konsola, print(), typy, zmienne
- 7 Pułapki dla początkujących

Geneza języka Python

1989 Guido van Rossum rozpoczyna pracę nad Pythonem (Centrum Wiskunde & Informatica, Holandia)

1991 Python 0.9.0 - pierwsza publiczna wersja

2000 Python 2.0 - list comprehensions, garbage collector

2008 Python 3.0 - niekompatybilny z 2.x, nowa era języka

2020 Koniec wsparcia dla Python 2 (EOL: 1 stycznia 2020)

2025 Python 3.14 - eksperymentalny free-threaded mode (no-GIL)

Nazwa od Monty Python's Flying Circus, nie od węża.

Dlaczego Python?

Czytelność

Składnia bliska pseudokodowi.
Minimum ceremonii składniowej.

Wszechstronność

Web, AI/ML, analiza danych, automatyzacja,
skrypty, GUI, embedded, IoT...

Ekosystem

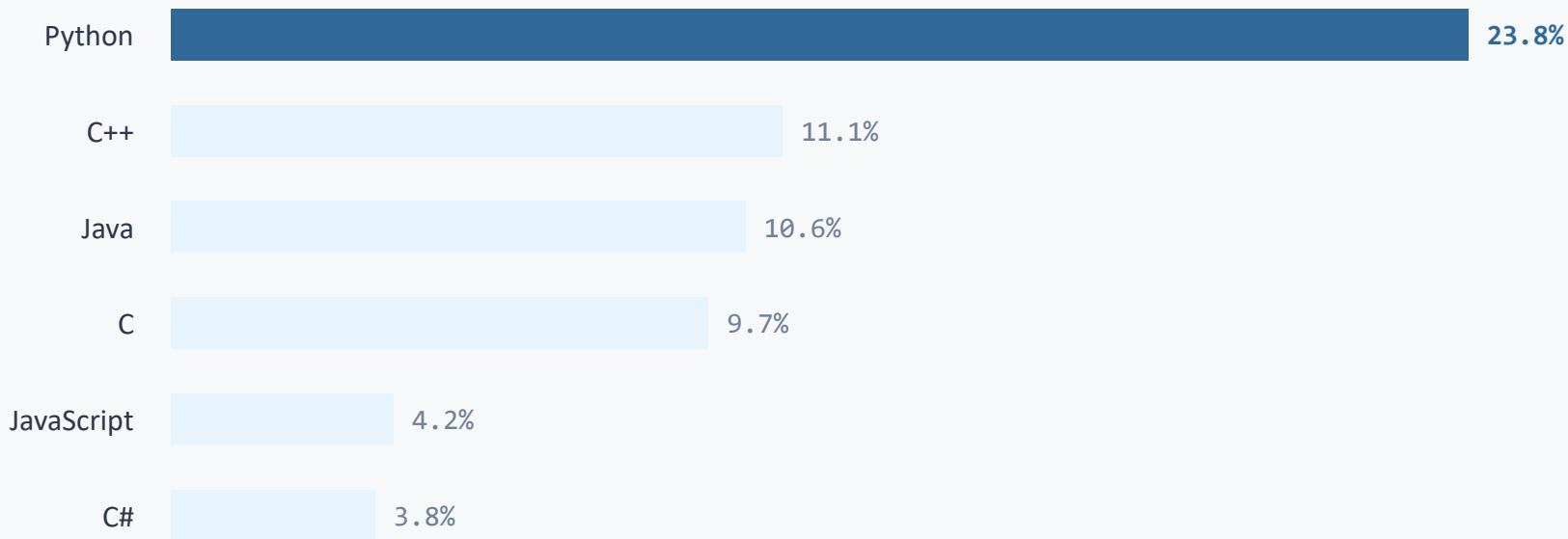
400 000+ pakietów na PyPI.
NumPy, Pandas, TensorFlow, Django...

Spoleczność

Ogromna baza wiedzy, Stack Overflow,
kursy, dokumentacja, meetupy.

Python na szczycie

TIOBE Index - ranking popularności języków programowania (2025)



Źródło: [tiobe.com](https://www.tiobe.com) (2025). Popularność mierzona liczbą zapytań w wyszukiwarkach.

Kluczowe cechy języka

Dynamiczne typowanie

Typ ustalany w trakcie działania, nie deklarowany w kodzie

Automatyczna pamięć

Garbage collector - brak ręcznego zwalniania pamięci

Wieloparadygmatowy

Proceduralny, obiektowy, funkcyjny w jednym

Interpretowany

Kompilacja do bytecode (.pyc), wykonanie na PVM

Wcięcia jako składnia

Bloki kodu definiowane przez wcięcia, nie przez { }

Baterie w zestawie

Bogata biblioteka standardowa (os, sys, json, re, math...)

Open source

Licencja PSF - wolny i darmowy, również komercyjnie

Interpreter vs kompilator

KOMPILATOR (C, C++, Rust)

- Cały kod → kod maszynowy przed uruchomieniem
- Wynik: plik wykonywalny (.exe, .out)
- Szybsze działanie programu
- Błędy wykrywane przy kompilacji

INTERPRETER (Python, Ruby, JS)

- Kod tłumaczony i wykonywany linia po linii*
- Brak pliku wykonywalnego
- Wolniejsze, ale łatwiejsze debugowanie
- Błędy wykrywane w trakcie wykonania

*Python: kompilacja do bytecode (.pyc) → wykonanie na maszynie wirtualnej (PVM)

CPython i inne implementacje

CPython

Referencyjna implementacja w C. Domyślna instalacja.

PyPy

JIT compiler - znacznie szybszy (5-10x) dla wielu zastosowań.

Jython

Python na JVM (Java Virtual Machine).

IronPython

Python na platformie .NET.

MicroPython

Zoptymalizowany dla mikrokontrolerów (ESP32, RPi Pico).

Na kursie: CPython 3.14 (najnowsza stabilna: 3.14.3).


```
>>> import this
```

The Zen of Python

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Readability counts.

Errors should never pass silently.

In the face of ambiguity, refuse the temptation to guess.

There should be one - and preferably only one - obvious way to do it.

Now is better than never.

Instalacja Pythona

- 1 Pobranie ze strony python.org/downloads
- 2 Zaznaczyć **Add Python to PATH** - to kluczowe!
- 3 Install Now (domyślna konfiguracja wystarczy)

Weryfikacja w terminalu:

```
python --version
```

```
Python 3.14.3
```

```
py --version          # Windows launcher
```

pip - menedżer pakietów

Pobiera pakiety z Python Package Index (PyPI) - pypi.org - 400 000+ projektów.

<code>pip install numpy</code>	<code># instalacja pakietu</code>
<code>pip install numpy==1.26.4</code>	<code># konkretna wersja</code>
<code>pip uninstall numpy</code>	<code># usunięcie</code>
<code>pip show numpy</code>	<code># informacje</code>
<code>pip list</code>	<code># lista zainstalowanych</code>
<code>pip freeze > requirements.txt</code>	<code># eksport do pliku</code>
<code>pip install -r requirements.txt</code>	<code># instalacja z pliku</code>
<code>pip install --upgrade numpy</code>	<code># aktualizacja</code>

Wirtualne środowisko - venv

Izolowane środowisko z własnymi pakietami. Każdy projekt może mieć inne wersje.

```
# Utworzenie wirtualnego środowiska
```

```
python -m venv venv
```

```
# Aktywacja (Windows / Linux)
```

```
venv\Scripts\activate
```

```
source venv/bin/activate
```

```
(venv) > pip install numpy
```

```
# Dezaktywacja
```

```
deactivate
```

Narzędzia pracy

IDE, edytory, notebooki i asystenci AI

Visual Studio Code

Darmowy, lekki, z ogromnym ekosystemem rozszerzeń. Rekomendowany edytor.

Python (Microsoft)

IntelliSense, debugger, linting, formatowanie

Pylint / Pylance

Analiza statyczna kodu, podpowiedzi błędów

autopep8 / black / ruff

Automatyczne formatowanie wg PEP 8

Code Runner

Szybkie uruchamianie: Ctrl+Alt+N

GitHub Copilot

Asystent AI -- podpowiedzi kodu w czasie rzeczywistym

Jupyter

Notebooki .ipynb bezpośrednio w VS Code

VS Code - przydatne skróty

Ctrl + Alt + N	Uruchom kod (Code Runner)
Ctrl + Shift + P	Command Palette
Ctrl + `	Terminal
Shift + Alt + F	Formatuj dokument (PEP 8)
Ctrl + Shift + M	Panel problemów
Ctrl + Shift + X	Rozszerzenia (Extensions)
F5	Debugger — uruchomienie z breakpointami
Ctrl + /	Zakomentuj / odkomentuj linię

Jupyter Notebook i Google Colab

Jupyter Notebook

- Kod + tekst + wykresy w jednym pliku (.ipynb)
- Popularny w data science i edukacji
- `pip install notebook`
- `jupyter notebook`

Google Colab

- Jupyter w chmurze -- nic nie trzeba instalować
- Darmowy dostęp do GPU/TPU
- Integracja z Google Drive
- Idealny do nauki i eksperymentów

Narzędzia AI w programowaniu

GitHub Copilot

Wtyczka VS Code. Podpowiedzi kodu w czasie pisania.
Tryb Chat, Edit, Agent.

ChatGPT / Claude

Wyjaśnianie koncepcji, generowanie kodu,
debugowanie, review.

Cursor / Windsurf

Edytory z wbudowanym AI.
Kod generowany w kontekście całego projektu.

AI to narzędzie, nie zastępstwo myślenia. Zawsze weryfikować wygenerowany kod!

PEP - Python Enhancement Proposals

Oficjalne dokumenty: propozycje zmian, konwencje, procesy. python.org/dev/peps/

PEP 8 Styl kodu Pythona — wcięcia, nazewnictwo, odstępy

PEP 20 Zen of Python (import this)

PEP 257 Docstring conventions — dokumentowanie kodu

PEP 484 Type hints — adnotacje typów

PEP 572 Walrus operator `:=` (Python 3.8)

PEP 634 Structural Pattern Matching -- `match/case` (3.10)

PEP 703 Making the GIL Optional (free-threaded, 3.13+)

PEP 8 - najważniejsze zasady

Dobrze

```
x = 42
name = "Alice"
my_function(arg1, arg2)
```

```
class MyClass:
    pass
```

```
MAX_SIZE = 100
```

```
# 4 spacje wcięcia
if x > 0:
    print(x)
```

Źle

```
x=42
Name = "Alice"
my_function( arg1,arg2 )
```

```
class my_class:
    pass
```

```
maxSize = 100
```

```
# tab + 2 spacje (mieszanie!)
if x > 0:
    print(x)
```

Pierwszy program

Konsola, print(), typy, zmienne

Konsola interaktywna

python (lub py) w terminalu. Każda linia jest natychmiast ewaluowana. Wyjście: quit().

```
>>> 2 + 3
5
>>> "hello" * 3
'hellohellohello'
>>> type(3.14)
<class 'float'>
>>> 10 / 3
3.3333333333333335
>>> 10 // 3
3
>>> 2 ** 100
1267650600228229401496703205376
```

Ciekawostka: int w Pythonie ma nieograniczony zakres — ogranicza go tylko RAM!

Pierwszy skrypt: hello.py

```
hello.py
```

```
# Pierwszy program w Pythonie
```

```
name = input("Jak masz na imie? ")  
print("Czesc,", name, "!")  
print(f"Twoje imie ma {len(name)} liter.")
```

```
$ python hello.py  
Jak masz na imie? Jan  
Czesc, Jan !  
Twoje imie ma 3 liter.
```

Funkcja print() -- szczegoly

```
print(*objects, sep=" ", end="\n", file=sys.stdout, flush=False)
```

```
print("A", "B", "C")
```

```
# A B C
```

```
print("A", "B", sep="-")
```

```
# A-B
```

```
print("A", end=" "); print("B")
```

```
# A B (w jednej linii)
```

```
print(f"{3.14159:.2f}")
```

```
# 3.14 (f-string)
```

sep

separator między arg.
(domyslnie spacja)

end

co dopisać na końcu
(domyslnie \n)

f-string

formatowanie w { }
od Python 3.6

Komentarze i docstringi

```
# To jest komentarz jednoliniowy
x = 42 # komentarz na końcu linii

"""
Docstring – wieloliniowy łańcuch znakowy.
Do dokumentowania modułów, klas i funkcji.
"""

def dodaj(a, b):
    """Zwraca sumę a i b."""
    return a + b

help(dodaj)
# Help on function dodaj: Zwraca sumę a i b.
```


Typy danych — przegląd

Typ	Przykład	Opis
<code>int</code>	42, -7, 0b1010, 0xFF	Całkowita (nieograniczony zakres!)
<code>float</code>	3.14, 2.0e-3, float('inf')	Zmiennoprzecinkowa (IEEE 754, 64-bit)
<code>bool</code>	True, False	Logiczny (podtyp int!)
<code>str</code>	'abc', "hello", f"{x}"	Łańcuch znakowy (niemutowalny)
<code>list</code>	[1, 2, 3]	Lista (mutowalna, uporządkowana)
<code>tuple</code>	(1, 2, 3)	Krotka (niemutowalna)
<code>dict</code>	{'a': 1, 'b': 2}	Słownik (klucz → wartość)
<code>set</code>	{1, 2, 3}	Zbiór (unikatowy, nieuporządkowany)
<code>None</code>	None	Brak wartości

Dynamiczne typowanie

```
x = 42  
print(type(x))  # <class 'int'>
```

```
x = "hello"  
print(type(x))  # <class 'str'>
```

```
x = [1, 2, 3]  
print(type(x))  # <class 'list'>
```

```
# Ta sama nazwa -> rozne obiekty.  
# Zmienna to etykieta,  
# NIE pudełko na dane!
```

Kluczowa idea:

Zmienna nie ma typu.
Obiekt ma typ.

Zmienna to tylko
referencja
do obiektu w pamięci.

id(x) zwraca adres
obektu w pamięci.

Wcięcia to nie dekoracja — to składnia!

IndentationError

```
if True:
print('tak')

for i in range(3):
    print(i)
    print('koniec')
```

Poprawnie

```
if True:
    print('tak')

for i in range(3):
    print(i)
    print('koniec')
```

Zawsze 4 spacje (nie tab!). Nie mieszać. Edytor: Tab = 4 spacje. Shift+Alt+F formatuje.

Pułapka: dzielenie i typy liczbowe

```
>>> 10 / 3                # dzielenie → zawsze float!
3.3333333333333335

>>> 10 // 3               # dzielenie całkowite
3

>>> 10 % 3                # reszta z dzielenia
1

>>> 0.1 + 0.2             # klasyczna pułapka IEEE 754
0.30000000000000004

>>> 0.1 + 0.2 == 0.3      # dlatego NIE porównywać ==
False

>>> round(0.1 + 0.2, 1) == 0.3
True
```

Do precyzyjnych obliczeń: moduł decimal lub fractions.

Pułapka: is vs ==

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b           # porównanie wartości
True
>>> a is b           # porównanie tożsamości (adresu!)
False

# Ale uwaga na optymalizacje Pythona:
>>> x = 256
>>> y = 256
>>> x is y           # True! (cache int -5..256)
True
>>> x = 257
>>> y = 257
>>> x is y           # False! (poza cache)
False
```

== porównuje wartość, **is** porównuje tożsamość obiektu (adres w pamięci).

Pułapka: mutowalna lista

```
>>> a = [1, 2, 3]
>>> b = a                # b wskazuje na TEN SAM obiekt!
>>> b.append(4)
>>> print(a)
[1, 2, 3, 4]

>>> c = a.copy()         # prawdziwa kopia (shallow)
>>> c.append(5)
>>> print(a)
[1, 2, 3, 4]
>>> print(c)
[1, 2, 3, 4, 5]
```

Przypisanie `b = a` nie kopiuje listy -- obie nazwy wskazują na ten sam obiekt.

Do kopiowania: `a.copy()`, `list(a)`, `a[:]` lub `copy.deepcopy(a)` dla zagnieżdżeń.

Python - to warto wiedzieć

```
>>> True + True + True      # bool to podtyp int!
3

>>> import antigravity      # otwiera xkcd w przeglądarce

>>> "Hello"[::-1]           # odwrócenie stringa
'olleH'

>>> a, b = b, a              # zamiana wartości bez tmp!

>>> x = 10
>>> 1 < x < 100              # łańcuchowe porównania!
True

>>> '_'.join(['a','b','c'])  # łączenie stringów
'a_b_c'
```

if `__name__ == "__main__"`

Pozwala odróżnić, czy plik jest uruchamiany bezpośrednio, czy importowany.

```
program.py
def powitanie(imie):
    """Wypisuje powitanie."""
    print(f"Czesc, {imie}!")

def main():
    imie = input("Podaj imie: ")
    powitanie(imie)

if __name__ == "__main__":
    main()
```

`python program.py` → `__name__ = "__main__"` → `main()` się uruchomi

`import program` → `__name__ = "program"` → `main()` się NIE uruchomi

Pułapka: string jest niemutowalny

```
>>> s = "Python"
>>> s[0] = 'J'
TypeError: 'str' object does not support item assignment
```

Trzeba stworzyć nowy string:

```
>>> s = "J" + s[1:]
>>> s
'Jython'
```

Niemutowalne (immutable): int, float, str, tuple, frozenset, bool

Mutowalne (mutable): list, dict, set, obiekty klas

Jeszcze kilka sztuczek

```
# Wielokrotne przypisanie
```

```
>>> x, y, z = 1, 2, 3
```

```
# Rozpakowywanie z *
```

```
>>> first, *rest = [1, 2, 3, 4, 5]
```

```
>>> rest
```

```
[2, 3, 4, 5]
```

```
# Mnożenie list (uwaga na pułapkę!)
```

```
>>> [0] * 5
```

```
[0, 0, 0, 0, 0]
```

```
# Warunek trójskładnikowy (ternary)
```

```
>>> x = 10
```

```
>>> wynik = "duzo" if x > 5 else "malo"
```

```
>>> wynik
```

```
'duzo'
```

Podsumowanie

- 1 Python - interpretowany, dynamicznie typowany, wieloparadygmatowy
- 2 Wszystko jest obiektem - zmienne to etykiety (referencje)
- 3 pip + venv - zarządzanie pakietami i izolacja projektów
- 4 VS Code + rozszerzenia - rekomendowane środowisko pracy
- 5 PEP 8 - snake_case, 4 spacje, czytelność ponad wszystko
- 6 Uwaga na pułapki: 0.1+0.2, is vs ==, mutowalna lista
- 7 if __name__ == "__main__" - dobra struktura programu